

Bloating Patterns + Step Pyramid

บวมบน · บวมล่าง · บวมกลาง · Dynamic Sizing · Step Pyramid vs Size Pyramid

จุดหักล้างความเชื่อเดิม: Martingale ทำได้ใน Grid เพราะรู้ทุนล่วงหน้า

แก่นของ PART นี้

Grid บวม หมายถึง inventory สะสมหนาแน่นในโซนใดโซนหนึ่ง — ไม่ใช่สิ่งผิดปกติ แต่ต้องเข้าใจ pattern และออกแบบล่วงหน้า Close System ทำให้เราสามารถ ใช้

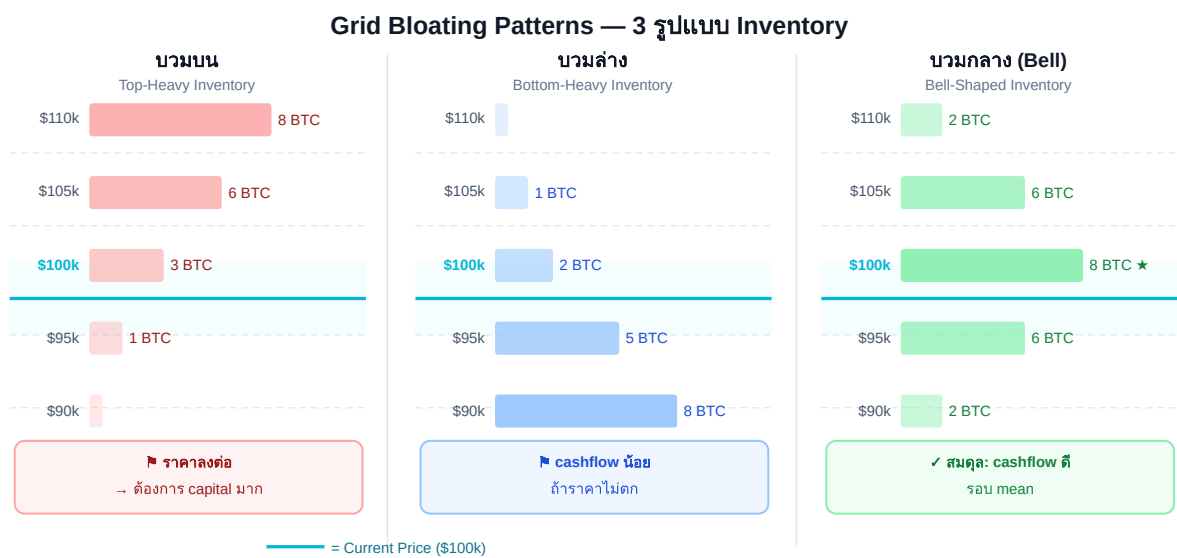
Martingale/Dynamic Sizing ได้อย่างปลอดภัย เพราะรู้ worst-case capital requirement แล้ว

2.1 Grid Bloating คืออะไร

เมื่อราคาอยู่ในโซนหนึ่งนานๆ grid ซื้อสะสม inventory ในโซนนั้น — เรียกว่า **grid บวม** ในโซนนั้น

ตัวอย่าง: BTC ลงจาก \$100k → \$90k แล้วแกว่งที่ \$90k–\$95k นาน 2 สัปดาห์

- Levels \$98k–\$92k ซื้อสะสม แต่ยังไม่ sell (ราคาไม่กลับมา)
- Levels \$92k–\$90k fill และ sell ชั่วๆ (cashflow ดี)
- ผลลัพธ์: inventory "บวม" ที่ระดับ \$90k–\$98k มี BTC ค้างอยู่มาก



ทั้ง 3 แบบมี P&L ใกล้เคียงกัน — ความต่างคือ risk distribution และ smoothness ของ cashflow

3 รูปแบบ bloating — P&L รวมใกล้เคียงกัน แต่ risk distribution และ cashflow timing ต่างกัน

2.2 บวมบน (Top-Heavy) — Inventory สะสมสูง

บวมบน (Top-Heavy)

inventory มากในโซนบน

ราคาสูง → ซื้อสูง รอขาย

เกิดเมื่อ: ราคา เคยสูง แล้วลงมา — grid ซื้อระหว่างทางลง แต่ราคาไม่กลับขึ้นไปขาย

สาเหตุ Top-Heavy:

1. เปิด grid ที่ราคาสูงกว่า mean ปัจจุบัน
2. BTC ดิ่งลง → grid ซื้อทุก level
3. ราคาหยุดแกว่งที่ต่ำกว่า zone กลาง

ผลกระทบ:

- + Grid levels ต่ำยังทำ cashflow ได้ปกติ
- Upper inventory ขาดทุน unrealized สูง
- ถ้าราคาพุ่งกลับ → ขายได้ทันที + กำไรมาก
- ถ้าราคาลงต่อ → inventory เพิ่มต่อ

แนวทางจัดการ:

- Option A: รอ (Close System — ถือไว้)
- Option B: ลด Q ของ upper levels ลง 50%
(ใช้ทุนน้อยลงในส่วนที่ "ดอย")
- Option C: เพิ่ม hedge (short perp ส่วนที่ upper inventory)

แม่ค้าผลไม้กับ grid ที่บวม

สามตัวเลือกข้างบนใช้ได้จริงทั้งสามทาง แต่ยังติดคำถามหนึ่งข้อ: **เมื่อไหร่ควรใช้ตัวไหน?**

ลองดูวิธีของคนที่เจอปัญหานี้มาก่อน grid trader หลายพันปี: แม่ค้าผลไม้ เวลาส้มในแผงค้างสต็อกเยอะ แม่ค้าไม่เคยปิดร้านหนี และไม่เคยรับซื้อส้มเพิ่มในราคาเดิมด้วย — เธอทำสองอย่างพร้อมกัน: **ลดราคารับซื้อลงอีกหน่อย** (อยากได้ของเพิ่มก็ต้องถูกจริงๆ) และ **ยอมขายออกไว้อีกหน่อย** ยิ่งสต็อกหนัก ยิ่งกด-ยิ่งยอม พอสต็อกกลับมาปกติ ราคารับซื้อ-ขายก็เลื่อนกลับที่เดิม

นั่นแหละคือคำตอบทั้งหมด และมันบีบเลือกกฎข้อเดียว:

ยิ่งถือของเต็มมือ จุดซื้อไม้ถัดไปยิ่งต้องไกลขึ้น

วัดความ "เต็มมือ" ด้วยสัดส่วนที่มีอยู่แล้วจาก Part 1A: ทุนที่ใช้ซื้อสะสมไปแล้ว (cumulative cost ในตาราง 1A.9) เทียบกับ C_floor (ทุนเต็มถ้าซื้อครบทุก level ถึง floor) — เรียกสั้นๆ ว่า "ถึงเต็มกี่ %" แล้วสามตัวเลือกข้างบนก็เลยเป็นทางเลือก กลายเป็นสเกลเดียว:

ถึงเต็ม (cumulative cost / C_floor)	ทำอะไร	ตรงกับ
ต่ำกว่า 30%	เล่นตามปกติ — ยังมีพื้นที่รับของ	Option A: รอ
30–70%	ขยับ step ฝั่งซื้อให้ห่างขึ้นตามสัดส่วนถึง (สูตรในกล่องด้านล่าง)	Option B: ลด Q / ขยาย step
เกิน 70%	หยุดรับของเพิ่ม แล้วปกป้องของที่มี	Option C: hedge (Part 1C)

สังเกตว่า **Step Pyramid** ในหัวข้อ 2.6 ใช้กฎข้อนี้จริงๆ — รวม Hurst regime (ตลาดเริ่ม trend) กับถึงเต็ม% (จากสูตรด้านบน) เป็นสูตรเดียว มันคือคำตอบเดียวกับที่งานวิจัยด้าน market making มืออาชีพคำนวณออกมาได้ (ดูอ่านต่อท้ายหัวข้อ) — จะเรียกมันว่า inventory skew หรือ "วิธีของแม่ค้าผลไม้" ก็ความหมายเดียวกัน

► สูตรสำหรับคนอยากตั้งเป็นตัวเลข (กดเปิด)

อ่านต่อ (ไม่จำเป็นต้องอ่านเพื่อใช้เล่นนี้): Avellaneda & Stoikov, "High-Frequency Trading in a Limit Order Book" (2008) — ต้นตำรับ inventory skew ของ market maker มืออาชีพ

2.3 บวมล่าง (Bottom-Heavy) — Inventory สะสมต่ำ

บวมล่าง (Bottom-Heavy)

inventory มากในโซนล่าง
ราคาลงมาก่อน กลับมา

เกิดเมื่อ: ราคา *ดิ่งลงไกล* สะสม inventory มาก แล้วค่อยๆ *ฟื้น* — levels ล่างๆ fill ซ้ำๆ บ่อย

สาเหตุ Bottom-Heavy:

1. BTC crash ลง 30%+ จาก entry
2. Grid ซื้อ level ล่างสุดทั้งหมด (Close System สำเร็จ)
3. ราคาฟื้นช้า แกว่งที่ bottom zone

ผลกระทบ:

- + Cashflow จาก lower zone ดี (รอบเยอะ)
- + ถ้าราคาพุ่งกลับ → ขายได้ทั้งหมด กำไรใหญ่
- Capital lock ในโซนล่างมาก
- Upper zone ว่าง → ไม่ได้ใช้ชั้นบน

แนวทางจัดการ:

- Option A: เพิ่ม grid levels ใหม่ที่โซนบน
(Front-load capital จาก Zone ที่ว่าง)
- Option B: นำ inventory ล่างขาย แล้วซื้อใหม่สูงกว่า
(Grid Reset — ดูใน Part 4)

2.4 บวมกลาง (Center-Heavy / Bell) — Ideal Pattern

บวมกลาง (Bell)

inventory หนาแน่น $\pm 1\sigma$

cashflow ดีที่สุด

เกิดเมื่อ: ราคาแกว่งรอบ mean อย่างสมดุล — Bell Curve Density (Part 1A) ออกแบบมาให้ได้ pattern นี้

ทำไม Bell Curve Pattern ดีที่สุด:

1. Inventory กระจายรอบ mean
2. Fill บ่อยสุดในโซนที่ราคาอยู่นานสุด ($\pm 1\sigma$)
3. Capital ไม่กระจุกตัวที่ extreme
4. ถ้าราคาออกนอก zone $\rightarrow$ inventory น้อย (step ใหญ่ไม่ fill บ่อย)

วิธีรักษา Bell Pattern:

- ใช้ Variable Step (bell curve density จาก Part 1A)
- Rebalance ทุกสัปดาห์ถ้า inventory เบี้ยวมาก
- ดู Hurst — ถ้า H ต่ำ (mean-reverting) pattern นี้ stable

2.5 Dynamic Sizing — จุดหักล้างความเชื่อเดิม

ทำไม Martingale ปลอดภัยใน Close System

ความเชื่อเดิม: Martingale อันตราย — เพิ่ม size เรื่อยๆ จนทุนหมด

ใน Close System: เราคำนวณ capital ทั้งหมดที่ต้องใช้ถึง floor ก่อน เปิด grid — Martingale แค่กำหนดว่าจะ allocate capital อย่างไรใน range ที่คำนวณไว้แล้ว

ตัวอย่าง: มี \$10,000 · Close System บอกว่า worst case ใช้ \$8,000 | Soft Martingale กำหนดว่าจาก \$8,000 นั้นจะใช้มากขึ้นที่ level ล่าง — แต่ไม่เกิน \$8,000 รวม

Dynamic Sizing Pseudocode:

```
def allocate_grid_capital(total_capital, n_levels, style="bell"):
    """
    Allocate capital across grid levels (all styles stay within
    total_capital)
    """
    if style == "equal":
        weights = [1.0 / n_levels] * n_levels

    elif style == "top_heavy":      # บวมบน: น้ำหนักมากที่บน (i=0 = level
    ใกล้ราคาปัจจุบันที่สุด)
        weights = [1.0 / (i + 1) for i in range(n_levels)]

    elif style == "bottom_heavy":  # บวมล่าง: น้ำหนักมากที่ล่าง (i มากขึ้น =
    ลีกลงไปถึง zone_low)
        weights = [1.0 / (n_levels - i) for i in range(n_levels)]

    elif style == "bell":          # บวมกลาง: Gaussian weights
        center = n_levels // 2
        weights = [math.exp(-0.5 * ((i - center) / (n_levels/4))**2)
                    for i in range(n_levels)]

    # Normalize weights to sum to 1
    total = sum(weights)
    weights = [w / total for w in weights]
```



```
# Allocate capital
capital_per_level = [total_capital * w for w in weights]
return capital_per_level

# ตัวอย่าง: $10,000 · 5 levels · bell style (center=2, n_levels/4=1.25)
# raw weights ก่อน normalize: [0.278, 0.726, 1.000, 0.726, 0.278] ← หน้า
#                             แนนกลาง, รวม = 3.008
# หลัง normalize (หารด้วย 3.008 แล้วคูณ $10,000):
#   → [$924, $2,414, $3,324, $2,414, $924] ✓ รวม = $10,000
```

2.6 Step Pyramid vs Size Pyramid

เมื่อตลาดมีแนวโน้ม (H เพิ่มขึ้น) — มีทางเลือก 2 แบบในการ "respond":

Strategy	วิธี	ผลต่อ Cashflow	ผลต่อ Inventory Risk
Size Pyramid	เพิ่ม Q ที่ level ลึก (martingale)	คงที่	สูงขึ้น (ซื้อมากถ้าลง)
Step Pyramid	เพิ่ม Step size แทน	ลดลงเล็กน้อย	ต่ำกว่า (fill น้อยกว่า)
Equal Weight	ไม่เปลี่ยนอะไร (baseline)	ลดลงตาม trend	สะสมเร็ว

Step Pyramid Implementation (สูตรรวม – canonical, ที่อื่นในเล่มอ้างมาที่นี่):

```
def compute_step(base_step, hurst, tank_fullness,
                 h_normal=0.50, h_max=0.58, trend_factor=0.5):
    """
    รวม 2 สัญญาณที่ทำให้ step ต้องห่างขึ้น:
    - Hurst สูง (trending, ดู Part 4) → เพิ่ม step
    - ถึงเต็ม% สูง (inventory skew, สูตรจาก §2.2) → เพิ่ม step อีกชั้น
    แทนที่ depth_multiplier แบบเดา (เวอร์ชันก่อนหน้า) ด้วยถึงเต็ม% จริง
    """
    hurst_ratio = min(1.0, max(0, (hurst - h_normal) / (h_max -
h_normal)))
    hurst_multiplier = 1 + hurst_ratio * trend_factor
    tank_multiplier = 1 + tank_fullness # ตรงกับสูตรใน §2.2: step × (1
+ ถึงเต็ม%)
    return base_step * hurst_multiplier * tank_multiplier

# ตัวอย่าง (ต่อจากตาราง 1A.9): base_step = $2,000 · Hurst = 0.55
# ถึงเต็มที่ level 3 = cumulative $3,826 / C_floor $8,460 = 0.452 (45.2%)
# hurst_ratio = (0.55-0.50)/(0.58-0.50) = 0.625
# hurst_multiplier = 1 + 0.625×0.5 = 1.3125
# tank_multiplier = 1 + 0.452 = 1.452
# step = $2,000 × 1.3125 × 1.452 ≈ $3,812 (ใหญ่ขึ้น → fill น้อยลง)
```

Step Pyramid: กำไรเท่าเดิม สบายใจกว่า

ทำไม Step Pyramid ดี: ในช่วง trend

- Size Pyramid: ซื้อมากขึ้นที่ราคาต่ำ → ถ้า trend กลับ กำไรมาก แต่ถ้า trend ต่อ → inventory massive
- Step Pyramid: ซื้อช้าลง (fill น้อยกว่า) → inventory สะสมช้า → ถ้า trend ต่อ ไม่ติดสต็อกมาก
- P&L รวมใกล้เคียงกัน แต่ **drawdown** ต่างกันมาก

2.6.1 TP ไม่ได้มีแบบเดียว — เชื้อหมุนฝั่งขาย (ปรับระยะ TP ได้เหมือนปรับ step ฝั่งซื้อ)

หัวข้อที่แล้วสอนขยับ step ฝั่งซื้อเมื่อตลาดเริ่ม trend — คำถามที่ควรถามต่อทันทีคือ: แล้วฝั่งขายล่ะ? ทำไม TP ต้องอยู่ที่ level ถัดไปเสมอ? ตลอดเล่มนี้ TP = "ขายที่ level ถัดไป" (1 step) เป็นค่าที่ตั้งตายตัวมาโดยตลอด — มันคือ default ที่ดี แต่ไม่ใช่ทางเลือกเดียว:

TP Policy	กลไก	เหมาะกับ Regime	Trade-off
1-Step (default)	ขายที่ level ถัดไปเสมอ (entry + step)	H ต่ำ (<0.50, mean-reverting) — ราคาแกว่งกลับบ่อย	รอบถี่ cashflow สม่ำเสมอ กำไรต่อรอบเล็ก
TP กว้าง (Sell-Side Pyramid)	ขยาย TP distance ตาม H — สัมพันธ์กับ Step Pyramid ฝั่งซื้อใน §2.6	H เริ่มขึ้น (0.50–0.58, CAUTION)	รอบน้อยลง กำไรต่อรอบใหญ่ขึ้น แต่ inventory ค้างรอนานกว่า
Partial TP	ขายบางส่วนที่ level ถัดไป ปลอยที่เหลือรอ level ถัดไปอีก	ไม่ผูก regime — เป็นทางเลือกลดความเสี่ยง all-or-nothing ล้วนๆ	ลดความเสี่ยงขายหมดเร็วเกินไป แต่ต้อง track quantity ที่เหลือต่อ level (ดูกล่องเตือน)
Trailing / Infinity	ไม่ตั้ง TP ตายตัว เลื่อนตามราคา — ดู Infinity Grid ใน Part 1D §1D.3	H สูงมาก เทรนด์ชัดเจน	เก็บ upside เทรนด์ได้เต็มที่ แต่เสีย cashflow สม่ำเสมอที่ 1-step ให้

ข้อควรจำ

ไม่มี TP policy ไหน "กำไรรวมมากกว่า" — ทุกแบบปรับแค่ **รูปทรง** ของ P&L (ความถี่รอบ, ขนาดกำไรต่อรอบ, drawdown, ความเสี่ยง inventory ค้าง) ไม่ใช่เพิ่ม expected value สุทธิ
ยังเป็นไปตาม zero-EV theorem ใน Part 0 §0.4 เหมือนเดิม

คำเตือน: Partial TP ต้องมี accounting เพิ่ม

Framework ใน Part 9 §9.2 (GridLevel) เก็บ qty เป็นค่าเดียวต่อ level และ filled เป็น boolean — ออกแบบมาสำหรับ all-or-nothing เท่านั้น ถ้าจะใช้ Partial TP จริง ต้องขยาย

GridLevel ให้ track "qty ที่ขายไปแล้ว" กับ "qty ที่เหลือ" แยกกันต่อ level ก่อน — เล่มนี้ยังไม่ implement ส่วนนั้น เป็นงานต่อยอดสำหรับใครอยากทำจริง

2.7 Running Example — Comparing 3 Bloating Patterns

 **Running Example: BTC ลงจาก \$100k → \$85k แล้วแกว่ง**

Setup: Zone \$80k–\$110k · Step \$2k · Capital \$15,000 · 3 strategies เปรียบกัน

Strategy	Inventory ที่ \$85k	Capital ที่ ใช้	Cashflow/วัน (ที่ \$83k–\$88k)	Profit ถ้า BTC กลับ \$100k
Equal Weight	0.08 BTC	\$7,200	\$36	+\$1,200
Bottom-Heavy (บวมล่าง)	0.14 BTC	\$12,320	\$54	+\$2,100
Bell Curve	0.06 BTC	\$5,400	\$28	+\$900

สังเกต: Bottom-Heavy ได้ทั้ง cashflow สูงสุดและ profit กลับมากที่สุด แต่ใช้ capital มากสุดด้วย

ถ้า BTC ลงต่อไป \$70k: Bottom-Heavy มี max loss สูงสุด | Bell Curve มี max loss ต่ำสุด

สรุป: เลือก style ตาม *conviction* — ถ้ามั่นใจ BTC จะกลับ → Bottom-Heavy | ถ้าไม่แน่ใจ → Bell Curve

2.8 แบบฝึกหัด

แบบฝึกหัด Part 2

- ▶ ข้อ 1: Grid บวมบน (Top-Heavy) มักเกิดใน market condition แบบไหน?
- ▶ ข้อ 2: ทำไม Size Pyramid ถึงเสี่ยงกว่า Step Pyramid ใน trending market?
- ▶ ข้อ 3: Bell Curve Grid density ให้ $Q_{center} = 0.02 \text{ BTC}$ · $Q_{extreme} = 0.005 \text{ BTC}$ · Step เท่ากัน \$2k — เปรียบ cashflow vs Equal Weight $Q=0.01 \text{ BTC}$ ทุก level
- ▶ ข้อ 4: ถ้า inventory ค้าง "บวมล่าง" มาก ควร Reset Grid หรือรอ?
- ▶ ข้อ 5: Hurst ปัจจุบัน $H = 0.53$ — ควรเลือก TP policy แบบไหนจาก §2.6.1 และทำไม?